

System Design Document (SDD)

**Project Name: LabTrack - University Laboratory
Equipment Management System**

1. Introduction and Overview of the System	Error! No bookmark name given.
1.1 System Overview	2
1.2 Core Objectives	2
2. Software Architecture & Justification	2
2.1 Backend Layered Architecture	2
2.2 Frontend Architecture (Single-Page Application)	2
2.3 Architectural Justifications	3
2.3.1 Why It Fits LabTrack Specifically	3
2.3.2 Key Advantages Gained	3
2.3.3 Comparison With Other Architecture	3
3. Technology Stack & Justification	Error! No bookmark name given.
3.1 Stack Overview	4
3.2 Technology Justifications	4
4. Database Design and Normalization	5
4.1 Tables	5
4.2 Relationships	5
4.3 Normalization	6
5. Object-Oriented Concepts	6
5.1 Encapsulation	6
5.2 Inheritance (Generalization/Specialization)	6
5.3 Polymorphism	7
5.4 Abstraction	7
6. Design Patterns	7
6.1 Overview	8
6.2 Singleton Pattern	8
6.3 Factory Pattern	8
6.4 State Pattern	9
6.5 Other Patterns Considered but Excluded	9
7. System Design Diagrams	Error! No bookmark name given.
7.1 UML Class Diagram	10
7.2 Entity-Relationship (ER) Diagram	10
7.3 Use Case Diagram	12

1. Introduction and Overview of the System

1.1 System Overview

LabTrack is a web application engineered to manage university scientific instruments, electronic testing kits, consumable supplies, and laboratory schedules. It replaces manual paper logbooks with a centralized online platform, unifying students, course instructors, laboratory technical officers (TOs), and administrators.

1.2 Core Objectives

- Centralize asset data to provide one reliable record for all physical laboratory equipment across departments.
- Prevent double-booking, schedule overlaps, and unauthorized access to restricted equipment through automated slot reservation checks.
- Improve equipment reliability by scheduling timely maintenance and tracking faulty items to prevent breakdowns during practical sessions.
- Ensure full accountability by maintaining complete logs of all borrowing events, returns, reported damages, and automated fine calculations.

2. Software Architecture & Justification

2.1 Backend Layered Architecture

The backend employs a layered architecture mapping the request flow through strict boundaries:

- **Routes (routes/)**: Declares API endpoints, wires up middleware, and maps HTTP methods to controller functions without embedding business logic.
- **Middleware (middleware/)**: Intercepts requests to verify JWTs (authMiddleware), enforce role-based access control (roleMiddleware), and handle global errors (errorHandler).
- **Controllers (controllers/)**: Thin handlers that extract request data, call the appropriate service function, and return the HTTP JSON response.
- **Services (services/)**: Contains all business logic, input validation, ownership checks, multi-step transactions, and executes parameterized SQL via pool.query().
- **Data Access (config/db.js)**: A single, shared PostgreSQL connection pool reused by all services.

2.2 Frontend Architecture (Single-Page Application)

The frontend is built as a React Single-Page Application (SPA) using React Router for client-side navigation.

- React maintains a virtual DOM to compute minimal UI differences during navigation, patching only the necessary real DOM nodes instead of requesting a new document from the server.
- Application state, including the logged-in user and JWT, is preserved in memory via AuthContext and backed by localStorage to survive page refreshes.

2.3 Architectural Justifications

2.3.1 Why It Fits LabTrack Specifically

- **Multiple, non-trivial business rules per role** — Reservation approval, availability checking, overdue fine calculation, and maintenance tracking all involve real logic — the Service layer gives this logic one home instead of scattering it across route handlers.
- **Multiple user roles sharing overlapping data** — Controllers stay thin and role-focused, while Services and Repositories handle the shared underlying data (equipment, reservations) consistently for all three roles.
- **Team of multiple developers** — Because each layer has a distinct responsibility, group members can work on different services (e.g. one on ReservationService, another on FineService) with minimal file conflicts.
- **Testability requirement** — Services can be unit-tested independently of Express and the database by mocking repositories, and repositories can be tested independently of business logic.

2.3.2 Key Advantages Gained

Advantage	How LabTrack Benefits
Separation of concerns	Controllers handle HTTP only; Services handle business rules; Repositories handle SQL — no mixing of responsibilities.
Maintainability	Changing fine-calculation logic touches only FineService, not the routes or SQL code.
Testability	Business logic (e.g. reservation conflict checks) can be unit tested independently of Express and the database, by mocking repositories.
Reusability	ReservationService can be reused by both the student-facing approval workflow and the admin monitoring views.
Scalability path	Individual services could later be extracted into separately deployable units if required — a natural next step, unlike a monolith with no internal boundaries.
Parallel development	Layers and modules map to different files/folders per feature, reducing merge conflicts between group members.
Clear onboarding	Any team member or grader can trace a request's path predictably: Route → Controller → Service → Repository.

2.3.3 Comparison With Other Architecture

- **Layered Architecture vs. Unstructured MVC:** An unstructured MVC approach would embed business logic (e.g., reservation rules, fine calculations) directly into route handlers alongside raw SQL. The layered architecture separates these concerns, making the system testable, maintainable, and easier for multiple developers to work on concurrently without conflicts.

- **Layered Architecture vs. Microservices:** Splitting the system into independent microservices would introduce massive overhead regarding inter-service communication and distributed data consistency. The layered monolith provides structural separation without the deployment complexity unsuitable for a four-week project timeline.
- **React SPA vs. Multi-Page Application (MPA):** A traditional MPA forces a complete DOM teardown and full page reload on every link click, which would destroy in-memory JavaScript state such as active sessions or unsubmitted equipment carts. The React SPA approach avoids this "DOM effect," preserving application state across navigations.

3. Technology Stack & Justification

3.1 Stack Overview

Layer	Technology	Role in the Project
Frontend	React 18 + Vite	Single-page UI, fast dev server and build
Routing	React Router v6	Client-side navigation between pages
HTTP client	Axios	Talks to the backend REST API
Backend	Node.js + Express.js	REST API server
Auth	JWT + bcrypt	Stateless login tokens, salted password hashes
Database	PostgreSQL (hosted on Neon)	Relational storage for all core data
DB driver	pg (node-postgres)	Raw parameterised SQL, connection pooling
Payments	Stripe (test/sandbox mode)	Simulated fine payment

3.2 Technology Justifications

- **React SPA over Multi-Page Applications (MPA):** A traditional MPA (such as plain HTML or PHP/EJS server-rendered pages) discards the entire DOM tree, sends a fresh HTTP request, and rebuilds the page completely from scratch upon every link click. This "DOM effect" wipes out all in-memory JavaScript state, which would cause the logged-in user, the JWT held in memory, and the student's reservation cart to be lost on every single navigation. A React SPA avoids this by intercepting navigation client-side, computing minimal differences, and patching only the necessary real DOM nodes. This ensures shared UI components (like the Navbar) are never unmounted and application state persists smoothly.
- **PostgreSQL over NoSQL (MongoDB):** The LabTrack domain is inherently relational; users, labs, equipment, reservations, and fines all reference each other using foreign keys. Enforcing this level of data integrity in a document store like MongoDB would mean writing manual application-level code to replicate what PostgreSQL handles natively through FOREIGN KEY constraints and multi-table joins.
- **Raw pg Driver over ORMs (Prisma/Sequelize/TypeORM):** While an ORM eliminates hand-written SQL, it hides the exact queries being executed and adds a second technology to learn on top of SQL itself. Utilizing the raw pg driver allows for explicit, parameterized SQL queries, keeping

the learning curve low and visibly demonstrating a fundamental understanding of the database layer appropriate for a university project.

- **Express.js over Heavy Frameworks (NestJS):** Express is intentionally minimal. Heavier backend frameworks bring in decorators, modules, and dependency injection designed to solve organizational problems for huge codebases and large corporate teams. Express, when combined with a manual layered folder structure, achieves the same separation of concerns with far less boilerplate.
- **End-to-End JavaScript:** Using JavaScript for both the frontend (React) and backend (Node.js) eliminates context-switching between languages and allows both sides of the application to share standard npm tooling.

4. Database Design and Normalization

4.1 Tables

Table	Key Columns	Purpose
users	user_id PK, email UNIQUE, role (ENUM)	Students, Technical Officers, Admins
departments	department_id PK, name UNIQUE	Academic departments
laboratories	lab_id PK, department_id FK, technical_officer_id FK	One lab has at most one officer
equipment_categories	category_id PK, name UNIQUE	Shared category list
equipment	equipment_id PK, category_id FK, lab_id FK, total/available_quantity, status (ENUM)	Inventory per lab
reservations	reservation_id PK, student_id FK, lab_id FK, status (ENUM)	One booking request
reservation_items	reservation_item_id PK, reservation_id FK, equipment_id FK, quantity_requested	The "cart" — many items per reservation
borrowing_records	record_id PK, reservation_id FK, issued_by FK, issued_at, returned_at	Actual collection/return event
finances	fine_id PK, record_id FK, student_id FK, amount, status (ENUM)	Overdue penalty
maintenance_tickets	ticket_id PK, equipment_id FK, reported_by FK, status (ENUM)	Fault reports

4.2 Relationships

- departments 1 — N laboratories
- users (Technical_Officer) 1 — 1 laboratories (a lab has at most one officer; the reverse is enforced in code, not by a DB constraint — see 8.4)

- equipment_categories 1 — N equipment; laboratories 1 — N equipment
- users (Student) 1 — N reservations; laboratories 1 — N reservations
- reservations 1 — N reservation_items — N — 1 equipment (the many-to-many between reservations and equipment, resolved with a junction table)
- reservations 1 — 1 borrowing_records (one issue/return record per reservation)
- borrowing_records 1 — N fines (schema allows more than one, though the current code only ever inserts one per late return)
- equipment 1 — N maintenance_tickets

4.3 Normalization

- **1NF:** Every column holds a single, atomic value, and every table is uniquely identified by a primary surrogate key (e.g., SERIAL).
- **2NF:** The schema does not utilize composite primary keys, automatically satisfying 2NF by eliminating partial dependencies on the key.
- **3NF:** Non-key columns depend exclusively on their table's primary key. Lookup data is omitted from base tables and retrieved via SQL JOIN operations at query time.
- **Deliberate Denormalization:** The fines table intentionally stores student_id directly, creating a transitive dependency (fines → borrowing_records → reservations → users). This controlled redundancy optimizes read performance, allowing the paymentService to filter a student's fines with a single-table scan instead of executing multi-table joins on every query.

5. Object-Oriented Concepts

The LabTrack system's domain model and architectural blueprint are heavily grounded in the four core Object-Oriented (OO) principles: Encapsulation, Inheritance, Polymorphism, and Abstraction. While the backend implementation utilizes functional Node.js modules, the conceptual object model—as mapped out in the system's UML Class Diagram—strictly adheres to these paradigms to organize business logic and entity relationships.

5.1 Encapsulation

Encapsulation restricts direct access to an object's internal state, requiring interactions to pass through well-defined interfaces or methods. LabTrack applies this principle across multiple layers of the system:

- **Domain Data Protection:** Core entities such as Equipment and Reservation do not allow arbitrary data modifications. Attributes like an equipment's availability or a reservation's status are updated exclusively through specific operational methods (e.g., updateQuantity(), approveReservation(), rejectReservation()).
- **State Encapsulation:** The system encapsulates strict lifecycle states using database-level ENUMs (UserRole, EquipmentStatus, ReservationStatus, MaintenanceStatus, and PaymentStatus). This prevents invalid data assignment (e.g., manually typing an unsupported status string) and hides the internal state transition complexity from the higher-level routing layers.

5.2 Inheritance (Generalization/Specialization)

Inheritance is utilized to establish hierarchical relationships, promoting code reuse by allowing specialized child classes to inherit attributes and behaviors from a generalized parent class.

- **User Hierarchy:** The base User class encapsulates universally shared attributes (id, name, email, passwordHash, role). This class specializes into three distinct subclasses: Student, TechnicalOfficer, and Admin. Each subclass inherits the base identity properties but introduces actor-specific methods. For example, the Student class possesses makeReservation() and payFine(), while the TechnicalOfficer class introduces issueEquipment() and createMaintenanceTicket().
- **Equipment Hierarchy:** The base Equipment class defines shared inventory attributes (id, name, quantity, status). It is extended into specialized subtypes:
 - ConsumableEquipment: Adds properties/methods for items that are depleted upon use (use(quantity)).
 - NonConsumableEquipment: Adds asset tracking behaviors (getSerialNumber()).
 - CalibratedEquipment: Adds temporal tracking for scientific accuracy (lastCalibrationDate, isCalibrationDue()).

5.3 Polymorphism

Polymorphism allows objects of different types to be treated as instances of the same class through a common interface, while executing their own specialized behaviors behind the scenes.

- **Dynamic Equipment Handling:** When the system processes an equipment reservation, the reservation service interacts with the abstract Equipment interface. However, the underlying validation rules behave polymorphically based on the concrete equipment subtype. For example, evaluating isAvailable() for ConsumableEquipment strictly checks the numeric available_quantity against the requested quantity. Conversely, evaluating isAvailable() for CalibratedEquipment may polymorphically require checking both the quantity and whether isCalibrationDue() evaluates to false before permitting the reservation.

5.4 Abstraction

Abstraction focuses on exposing only the essential features of an object or system while hiding the underlying implementation details and complexity.

- **Architectural Abstraction:** The system's layered architecture serves as a macro-level abstraction. Controllers are completely abstracted away from raw SQL queries; they simply call reservationService.approve(id) without needing to know the complex multi-table JOIN operations, database pooling mechanics, or transaction rollbacks occurring within the service layer.
- **Object Creation Abstraction:** The EquipmentFactory abstracts the complex instantiation logic of the various equipment subtypes. The core application requests a new equipment object by providing basic data, and the factory obscures the decision-making process of determining whether to output a ConsumableEquipment or

6. Design Patterns

6.1 Overview

Design patterns are reusable approaches for solving common software design problems. LabTrack uses three main design patterns to ensure a robust, maintainable architecture:

1. Singleton Pattern
2. Factory Pattern
3. State Pattern

6.2 Singleton Pattern

- **Purpose:** The Singleton pattern ensures that a shared resource has a single shared instance within the application.
- **LabTrack Usage:** The Singleton concept is utilized for the PostgreSQL connection pool. Instead of creating a new database pool within every repository, the application instantiates and utilizes a single, shared pool.
- **Why it fits:** A database connection pool is a shared application resource. Creating unnecessary pools across modules wastes database connections and degrades application resources. The Singleton approach ensures centralized connection-management configuration and clear separation of responsibilities.
- **Comparison with Comparable Patterns:**
 - *Singleton vs. Prototype / Instance-Per-Request:* The Prototype pattern generates a new instance every time a resource is requested. If applied to database connections, the system would rapidly exhaust database connection limits and crash under load. The Singleton pattern guarantees one controlled, global access point to a single pooled instance, providing superior memory and connection management.

6.3 Factory Pattern

- **Purpose:** The Factory pattern centralizes object creation and separates the decision about which concrete class to instantiate from the business logic that uses the resulting object.
- **LabTrack Usage:** The Factory pattern is used to create different equipment objects. Instead of the rest of the application directly deciding which equipment class to instantiate, the EquipmentFactory handles object creation based on registration data (e.g., `const equipment = EquipmentFactory.create(type, data);`).
- **Why it fits:** It centralizes object creation, reduces repeated creation logic across the application, makes adding new equipment types easier, and keeps object creation completely separate from business logic workflows such as reservation and maintenance.
- **Comparison with Comparable Patterns:**
 - *Factory vs. Direct Instantiation (Constructor Calls):* Direct instantiation tightly couples core business workflows to concrete classes. The Factory pattern abstracts this, allowing the system to scale and support new equipment subtypes dynamically without altering existing reservation logic.

- *Factory vs. Builder Pattern:* The Builder pattern is designed for constructing highly complex objects step-by-step. LabTrack's equipment objects do not require complex, multi-step assembly; rather, they require dynamic subtype selection based on input data. Therefore, the Factory pattern is a more precise and lightweight fit for resolving these polymorphic subtypes.

6.4 State Pattern

- **Purpose:** The State pattern allows an object to behave differently depending on its current internal state.
- **LabTrack Usage:** Equipment goes through distinct operational states including AVAILABLE, RESERVED, BORROWED, MAINTENANCE, and DECOMMISSIONED. The equipment's operational state strictly affects what operations are allowed. For example, equipment under maintenance is automatically prevented from being available for ordinary reservation.
- **Why it fits:** It prevents invalid operations, improves system reliability, and cleanly organizes business rules regarding lifecycle behavior rather than scattering state checks throughout the application.
- **Comparison with Comparable Patterns:**
 - *State vs. Strategy Pattern:* The Strategy pattern is used to swap interchangeable algorithms at runtime, such as swapping out different fine-calculation rules. The State pattern restricts behaviors and capabilities based on an internal lifecycle transition. Because physical equipment naturally follows a strict lifecycle, the State pattern is the correct structural choice.
 - *State vs. Conditional Flags:* Relying on loose boolean conditional flags across various service files makes the code fragile and difficult to test. The State pattern centralizes these transition rules securely, ensuring predictable state-specific behavior.

6.5 Other Patterns Considered but Excluded

The following patterns were evaluated but excluded as core patterns to avoid unnecessary complexity within the current documented scope:

- **Observer Pattern:** Defines a one-to-many relationship to notify components of state changes. LabTrack utilizes a straightforward notification service abstraction; implementing a full event-driven Observer structure would add complexity not strictly required by the current scope.
- **Strategy Pattern:** Allows algorithms to be selected at runtime. LabTrack utilizes a fixed automated fine-calculation rule set rather than interchangeable fine policies, making the Strategy pattern unnecessary at this stage.
- **Decorator Pattern:** Adds behaviors dynamically without changing original classes. LabTrack's layered architecture sufficiently organizes operations without needing dynamic runtime wrappers for domain objects.

7. System Design Diagrams

This section presents the three core UML diagrams produced for LabTrack: the class diagram, the entity-relationship (ER) diagram, and the use case diagram. Together they describe the system's object model, its persisted data structure, and the functionality available to each user role.

7.1 UML Class Diagram

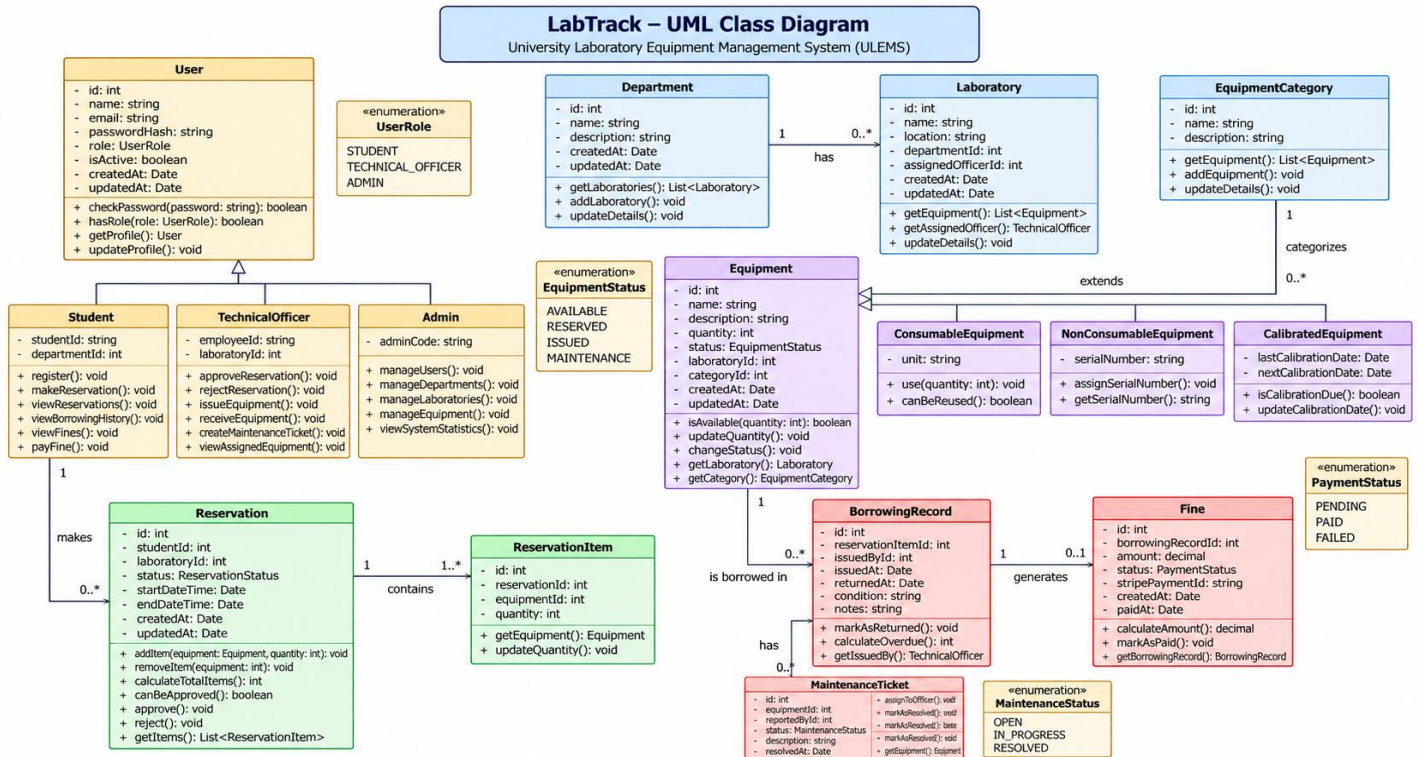


Figure 1 — LabTrack UML Class Diagram

This diagram models the system's object structure. A User specialises into Student, TechnicalOfficer and Admin, each with role-specific operations. A Department contains Laboratories, which hold Equipment classified by EquipmentCategory and specialised into ConsumableEquipment, NonConsumableEquipment and CalibratedEquipment. Students create Reservations made up of ReservationItems; approved reservations become BorrowingRecords, which can generate a Fine on late return. Equipment faults are tracked through MaintenanceTicket. The class diagram therefore mirrors the layered architecture and design patterns described earlier — equipment subtypes reflect the planned Factory pattern, and the UserRole, EquipmentStatus, MaintenanceStatus and PaymentStatus enumerations reflect the State-driven lifecycle rules discussed in Section 7.

7.2 Entity-Relationship (ER) Diagram

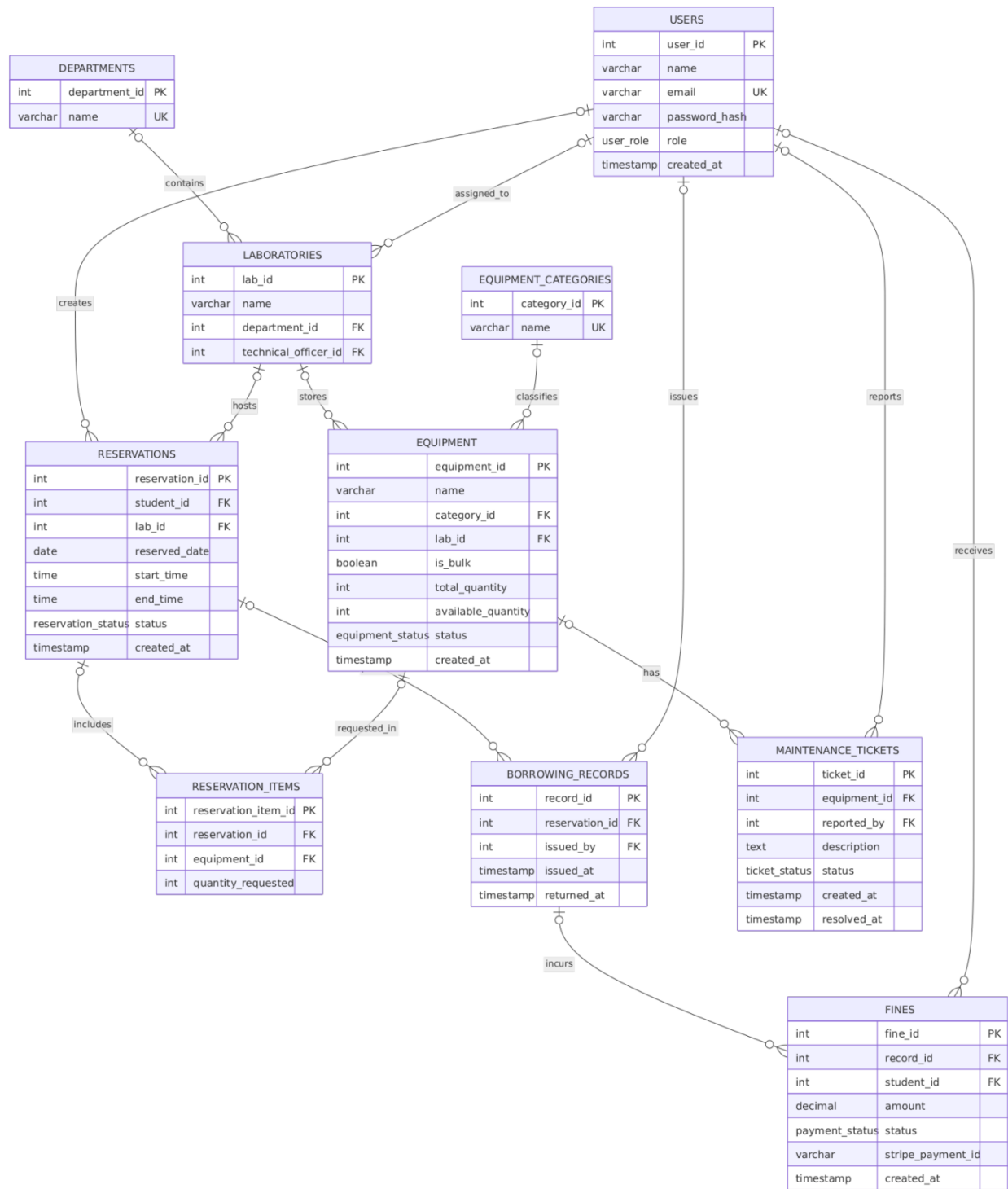


Figure 2 — LabTrack Entity-Relationship Diagram

This diagram shows how the class model above is persisted in PostgreSQL. USERS, DEPARTMENTS and LABORATORIES form the organisational structure, with LABORATORIES linked to an assigned technical officer. EQUIPMENT belongs to a laboratory and an equipment category, and is requested through RESERVATIONS and their RESERVATION_ITEMS (the junction table resolving the many-to-

many relationship between reservations and equipment). Approved reservations become BORROWING_RECORDS, which can incur FINES, while MAINTENANCE_TICKETS track faults reported against equipment. This is the concrete schema referenced in Section 8 (Database Schema and Normalization), including the foreign keys, ENUM-typed status columns, and the deliberate denormalization on the fines table.

7.3 Use Case Diagram

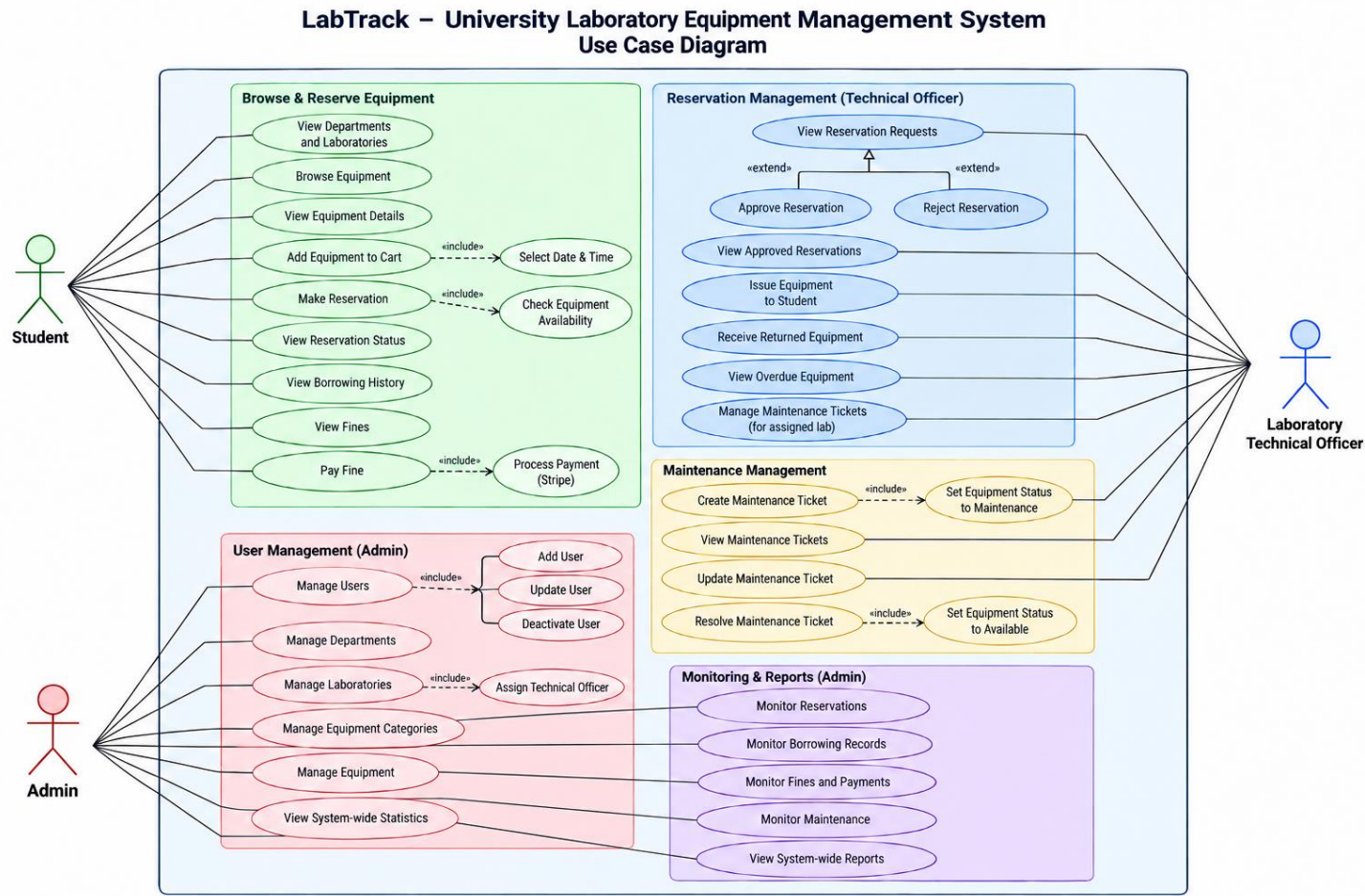


Figure 3 — LabTrack Use Case Diagram

This diagram summarize system functionality by actor. Students can browse departments and laboratories, reserve equipment, and manage their own reservations, borrowing history and fines. Laboratory Technical Officers approve or reject reservations, issue and receive equipment, and manage maintenance tickets for their assigned lab. Admins manage users, departments, laboratories and equipment categories, and monitor reservations, borrowing, fines and maintenance system-wide. The «include» and «extend» relationships capture supporting steps such as checking availability before making a reservation, processing a Stripe payment when paying a fine, and updating equipment status automatically when a maintenance ticket is created or resolved, consistent with the role-based access control described in Section 1.4 and the endpoint access rules in Section 4.2.